



عملي

3

13

1950 sp

كلية الهندسة المعلوماتية

السنة الرابعة

MULTITASKING AND PROCESSES

عملي مشترك



RB Informatics; 2025/11/4

نظم التشغيل

ذكرنا في المحاضرة الأولى أن من مميزات نظام لينكس تعدد المهام، وفي هذه المحاضرة سنتعلم عنها أكثر وكيفية إدارتها .

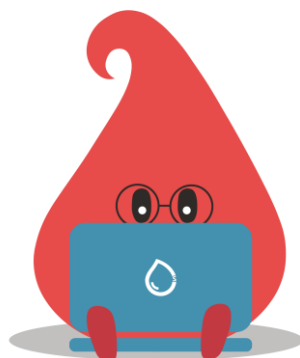
Multitasking and Processes

■ مفهوم تعدد المهام والعمليات :

إن أنظمة التشغيل الحديثة عادةً ما تكون متعددة المهام، بمعنى أنها تخلق وهم تنفيذ أكثر من عملية في نفس الوقت من خلال الانتقال السريع بين البرامج التي يتم تنفيذها. تقوم نواة لينكس بإدارة هذا عبر استخدام العمليات (Processes) وهي الطريقة التي يستخدمها لينكس لتنظيم البرامج المختلفة التي تنتظر دورها في وحدة المعالجة المركزية (CPU) .

■ مراحل الإقلاع بشكل مبسط

عند تشغيل الحاسوب، يبدأ الـ BIOS بالإقلاع أولاً، حيث يقوم بفحص العتاد والتأكد من سلامته. وإذا لم يكتشف أي مشكلات، يبحث بعد ذلك عن ملف إقلاع نظام التشغيل ويحمّله إلى ذاكرة RAM . يقوم بعدها المعالج بتنفيذ ملف الإقلاع، والذي بدوره يعمل على تحميل نواة النظام (kernel) إلى الذاكرة أيضاً. وعقب اكتمال تحميل النواة، تقوم النواة بتشغيل أول عملية في النظام، وهي systemd (الذي كان يُعرف سابقاً بـ init) بعد ذلك يتولى systemd تشغيل مجموعة من سكريبتات Shell Scripts الموجودة داخل مجلد /etc والمعروفة باسم Init Scripts، والتي تُعنى ببدء خدمات النظام وتشغيلها (initializing) .



إدارة العمليات (PROCESS MANAGEMENT)

- العديد من الخدمات في النظام يتم تنفيذها على شكل برامج Daemon، وهي برامج تعمل في الخلفية وتقوم بمهامها دون وجود واجهة استخدام. لذا حتى لو لم يكن أي مستخدم قد سجل الدخول، يبقى النظام منشغلاً بتنفيذ مهامه الروتينية.
- يحتفظ نواة النظام بمعلومات عن كل عملية للحفاظ على التنظيم. فعلى سبيل المثال، يتم تخصيص رقم لكل عملية يُسمى PID (معرف العملية). يتم إعطاء الـ PID بترتيب تصاعدي، حيث يحصل برنامج init دائماً على PID 1.
- كما يحتفظ النظام بمعلومات عن الذاكرة المخصصة لكل عملية، بالإضافة إلى حالة استعداد العملية لاستئناف التنفيذ. وكما هو الحال مع الملفات، فإن العمليات لها مالكون (owners)، ولها User IDs و Effective User IDs وما إلى ذلك.
- جميع العمليات يمكن رؤيتها في المسار /proc

Thread Vs Processes

- البرنامج (program) :
هو عبارة عن شيء ثابت static، عند تنفيذه يتم تحميله إلى الـ RAM عندها يصبح process
- العملية (Process) :
هي وحدة تنفيذ مستقلة لها مساحة الذاكرة الخاصة بها.
- المسار (Thread) :
هو عملية خفيفة جزئية (lightweight subprocess) تعمل داخل عملية (process) أخرى وتشارك نفس الذاكرة مع بقية الخيوط داخل العملية نفسها.
- يدعم لينكس تعدد المسارات (multithreading) من خلال مكتبة pthread ونظام إدارة المسارات داخل النواة.
- تستخدم المسارات بشكل كبير في لينكس لتحسين الأداء والاستجابة، خصوصاً على الأنظمة متعددة الأنوية (multi-core systems). فهي تسمح لأجزاء متعددة من البرنامج بالعمل بالتوازي داخل نفس العملية.



في القلب طاقة كالرياح تدفعنا نحو النجاح
مهما أثقلنا بالجراح شفائنا يكمل في الفلاح

■ أمثلة عن ال Threads في لينكس

1. Apache web server

- الاستخدام : يقوم Apache بإنشاء مجموعة من خيوط العمل (worker threads) لمعالجة طلبات HTTP الواردة.

- الفائدة : يسمح بالتعامل مع عدة اتصالات من العملاء في نفس الوقت دون إنشاء عملية جديدة لكل اتصال.

2. GNOME Desktop Environment

- الاستخدام : تطبيقات GNOME مثل مدير الملفات Nautilus تستخدم ال threads لتحميل المجلدات والصور المصغرة في الخلفية.

- الفائدة : الحفاظ على واجهة المستخدم سريعة الاستجابة أثناء القيام بعمليات الملفات.

3. System Daemons

- الاستخدام : خدمات الخلفية مثل udev و systemd تستخدم الخيوط (threads) لمراقبة أحداث العتاد وإدارة الخدمات.

- الفائدة : تنفيذ متعدد المهام بكفاءة دون تعطيل العملية الأساسية.

4. Database Servers (e.g. MySQL, PostgreSQL)

- الاستخدام : تستخدم ال threads للتعامل مع عدة استعلامات واتصالات في الوقت نفسه.
- الفائدة : تحسين الإنتاجية وتقليل التأخير في عمليات قواعد البيانات.

العمليات على Processes

1. عرض العمليات processes :

- يمكن عرض العمليات في لينكس باستخدام أدوات مثل ps و top
- يعرض الأمر ps لقطة ثابتة (static snapshot) للعمليات الحالية، بينما يوفر الأمر top تحديثات حية (real-time) ومستمرة.
- تعرض هذه الأدوات معلومات مثل استخدام المعالج (CPU) ، والذاكرة، وحالة العملية، وغيرها.
- يُعد الأمر top مفيدًا جدًا في مراقبة أداء النظام وتحديد العمليات التي تستهلك موارد كبيرة.

Knowledge is an art
requiring creativity and constant practice.



■ حالات العمليات (PROCESS STATES)

الوصف	الرمز	اسم الحالة
العملية تعمل حالياً أو جاهزة للتشغيل وتستخدم المعالج.	R	Running
العملية تنتظر حدثاً ما (مثل I/O), ويمكن مقاطعتها بواسطة الإشارات .	S	Sleeping (Interruptible)
العملية تنتظر مورداً ولا يمكن مقاطعتها (يحدث عادة أثناء عمليات الإدخال/الإخراج منخفضة المستوى الخاصة بالنواة).	D	Sleeping (Uninterruptible)
تم إيقاف العملية بواسطة إشارة (مثل SIGSTOP), ويمكن استئنافها.	T	Stopped
انتهت العملية ولكن ما زالت في جدول العمليات, وتنتظر تنظيفها بواسطة العملية الأب.	Z	Zombie
العملية لا تقوم بأي عمل؛ تستخدم عادة لعمليات النواة الخاملة.	I	Idle
العملية تحت التتبع أو التصحيح بواسطة عملية أخرى (مثل gdb)	t	Traced/Debugging

■ عرض العمليات باستخدام الأمر ps:

```
rbcs@tasneem-VMware-Virtual-Platform:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  5.0  0.3 22984 13936 ?        Ss   14:15   0:05 /sbin/init sp
root         2  0.0  0.0     0     0 ?        S    14:15   0:00 [kthreadd]
root         3  0.0  0.0     0     0 ?        S    14:15   0:00 [pool_workque
root         4  0.0  0.0     0     0 ?        I<   14:15   0:00 [kworker/R-rc
root         5  0.0  0.0     0     0 ?        I<   14:15   0:00 [kworker/R-sy
root         6  0.0  0.0     0     0 ?        I<   14:15   0:00 [kworker/R-kv
root         7  0.0  0.0     0     0 ?        I<   14:15   0:00 [kworker/R-sl
root         8  0.0  0.0     0     0 ?        I<   14:15   0:00 [kworker/R-ne
root         9  0.0  0.0     0     0 ?        I    14:15   0:00 [kworker/0:0-
root        10  0.0  0.0     0     0 ?        I    14:15   0:00 [kworker/0:1-
root        11  0.0  0.0     0     0 ?        I<   14:15   0:00 [kworker/0:0H
root        12  0.0  0.0     0     0 ?        I    14:15   0:00 [kworker/u512
root        13  0.0  0.0     0     0 ?        I<   14:15   0:00 [kworker/R-mm
root        14  0.0  0.0     0     0 ?        I    14:15   0:00 [rcu_tasks_kt
root        15  0.0  0.0     0     0 ?        I    14:15   0:00 [rcu_tasks_ru
root        16  0.0  0.0     0     0 ?        I    14:15   0:00 [rcu_tasks_tr
root        17  0.1  0.0     0     0 ?        S    14:15   0:00 [ksoftirqd/0]
root        18  0.7  0.0     0     0 ?        I    14:15   0:00 [rcu_preempt]
root        19  0.0  0.0     0     0 ?        S    14:15   0:00 [rcu_exp_par_
root        20  0.0  0.0     0     0 ?        S    14:15   0:00 [rcu_exp_gp_k
```

```

rbc@tasneem-VMware-Virtual-Platform:~$ ps
  PID TTY          TIME CMD
 2845 pts/0    00:00:00 bash
 2855 pts/0    00:00:00 ps
rbc@tasneem-VMware-Virtual-Platform:~$ ps x
  PID TTY          STAT TIME   COMMAND
 1864 ?        Ss     0:01   /usr/lib/systemd/systemd --user
 1865 ?        S       0:00   (sd-pam)
 1879 ?        S<sl    0:00   /usr/bin/pipewire
 1880 ?        Ssl     0:00   /usr/bin/pipewire -c filter-chain.conf
 1889 ?        S<sl    0:00   /usr/bin/wireplumber
 1890 ?        S<sl    0:00   /usr/bin/pipewire-pulse
 1891 ?        Ssl     0:00   /usr/bin/gnome-keyring-daemon --foreground --compon
 1892 ?        Ss      0:00   /usr/bin/dbus-daemon --session --address=systemd: -
 1942 ?        Ssl     0:00   /usr/libexec/xdg-document-portal
 1943 tty2      Ssl+    0:00   /usr/libexec/gdm-wayland-session env GNOME_SHELL_SE
 1947 tty2      Sl+     0:00   /usr/libexec/gnome-session-binary --session=ubuntu
 1959 ?        Ssl     0:00   /usr/libexec/xdg-permission-store
 2013 ?        Ssl     0:00   /usr/libexec/gcr-ssh-agent --base-dir /run/user/100
 2014 ?        Ssl     0:00   /usr/libexec/gnome-session-ctl --monitor
 2024 ?        Ssl     0:00   /usr/libexec/gvfsd
 2036 ?        Sl      0:00   /usr/libexec/gvfsd-fuse /run/user/1002/gvfs -f

```

شرح الفرق بين الأوامر السابقة:

أولاً شرح الخيارات:

- a: عرض عمليات جميع المستخدمين
- u: عرض التفاصيل بطريقة user-oriented (يعرض حقول %CPU, %MEM,)
- x: عرض العمليات غير المرتبطة بترمينال

ثانياً شرح الفرق بين الأوامر:

- ps: عمليات المستخدم الحالي المرتبطة بالترمينال فقط
- ps x: عمليات المستخدم الحالي سواء لها ترمينال أو لا
- ps aux: جميع العمليات على النظام لكل المستخدمين

■ شرح عناوين أعمدة ps:

عند تشغيل ps بأسلوب BSD مثل: ps aux ستظهر أعمدة تمثل معلومات العملية

المعنى	عنوان العمود في التعليمات
ال user ID لمالك العملية	USER
رقم العملية	PID
نسبة استهلاك المعالج	%CPU
نسبة استهلاك الذاكرة	%MEM
حجم الذاكرة الافتراضية	VSZ
حجم الذاكرة الحقيقية بال KB	RSS

الطرفية التي تعمل عليها العملية, عندما تكون قيمتها ؟ تعني أن العملية تعمل بدون أي واجهة طرفية	TTY
حالة العملية	STAT
الوقت التي بدأت فيه العملية, إذا كانت في نفس اليوم تعرض الساعة التي بدأت فيها العملية, وإلا يُعرض التاريخ	START
الوقت الذي استغرقه المعالج لتنفيذ العملية	TIME
الأمر الذي شغل العملية	COMMAND

■ عرض العمليات باستخدام الأمر top

يعرض الأمر top قائمة العمليات مع تحديث مستمر كل ثانيتين تقريباً.

```
rbcs@tasneem-VMware-Virtual-Platform:~$ top

top - 15:06:47 up 39 min, 1 user, load average: 0.08, 0.04, 0.05
Tasks: 293 total, 1 running, 292 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.3 us, 1.2 sy, 0.0 ni, 98.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3867.7 total, 2039.4 free, 1115.7 used, 973.2 buff/cache
MiB Swap: 3729.0 total, 3729.0 free, 0.0 used. 2752.0 avail Mem

  PID USER      PR  NI   VIRT   RES    SHR S  %CPU  %MEM    TIME+  COMMAND
 2090 rbcs       20   0 3934196 385160 142272 S   2.0   9.7   0:25.99 gnome-s+
 2833 rbcs       20   0 553848 52948  41972 S   0.7   1.3   0:02.44 gnome-t+
 3262 rbcs       20   0 3080184 60640  46628 S   0.7   1.5   0:00.42 gjs
    28 root        20   0      0      0      0 I   0.3   0.0   0:01.78 kworker+
 2544 rbcs       20   0 310760 6764   6124 S   0.3   0.2   0:00.04 gvfs-gp+
 2761 rbcs       20   0 2736340 27220  22100 S   0.3   0.7   0:00.13 gjs
 3187 root        20   0      0      0      0 I   0.3   0.0   0:00.09 kworker+
 3303 rbcs       20   0 14536   5848   3672 R   0.3   0.1   0:00.05 top
    1 root        20   0 22984  13936  9328 S   0.0   0.4   0:05.78 systemd
    2 root        20   0      0      0      0 S   0.0   0.0   0:00.06 kthreadd
    3 root        20   0      0      0      0 S   0.0   0.0   0:00.00 pool_wo+
    4 root        0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker+
    5 root        0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker+
    6 root        0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker+
```



شرح المعلومات:

معلومات النظام – سطر ال top	
الحقل	المعنى
Time	الوقت الحالي في النظام أثناء تشغيل أمر top.
Uptime	المدة التي بقي فيها النظام يعمل منذ آخر إعادة تشغيل.
Users	عدد المستخدمين المتصلين حالياً بالنظام.
متوسط الحمل Load Average	وهو ثلاث أرقام تمثل معدل الحمل قبل (من اليسار لليمين): دقيقة – 5 دقائق – 15 دقيقة وهو عدد العمليات التي تنتظر تنفيذها (قياس الضغط على المعالج).

استهلاك المعالج	
الحقل	المعنى
us (user processes)	نسبة الوقت الذي يستخدمه المعالج لتنفيذ عمليات المستخدم (البرامج العادية).
sy (system processes)	نسبة الوقت المستخدم لتنفيذ عمليات النظام (kernel).
ni (nice)	العمليات التي تم تغيير أولويتها بواسطة قيمة nice.
id (idle)	نسبة الوقت الذي يكون فيه المعالج خاملاً (بدون عمل).
wa (wait)	نسبة الوقت الذي ينتظر فيه المعالج عمليات الإدخال/الإخراج (مثل القرص).
hi (hardware interrupts)	وقت معالجة مقاطعات العتاد (مثل أجهزة الإدخال/الإخراج).
si (software interrupts)	وقت معالجة مقاطعات البرمجيات (soft IRQ).
st (stolen time)	الوقت "المسروق" من الـ CPU بواسطة جهاز افتراضي آخر (يظهر في الأنظمة الافتراضية).



استهلاك الذاكرة

الحقل	المعنى
total	حجم الذاكرة RAM الكلي في النظام.
Free	الذاكرة غير المستخدمة (المتاحة).
Used	الذاكرة التي يتم استخدامها
buff/cache	ذاكرة يستخدمها النظام للتخزين المؤقت وتسريع الأداء.
swap	جزء من القرص يُستخدم عند امتلاء RAM

جدول التعليلة

اسم العمود	المعنى
PR (priority)	أولوية العملية في جدول الجدولة.
NI (nice value)	تعديلها يؤدي لتعديل أولوية تنفيذ العملية (سيتم شرحها تالياً)
VIRT (virtual memory)	إجمالي الذاكرة الافتراضية التي تطلبها العملية (RAM + Swap)
RES (Resident memory)	حجم الذاكرة الحقيقية (RAM) المستخدمة فعلياً من قبل العملية
SHR (shared memory)	الجزء من الذاكرة المشتركة مع عمليات أخرى.
S (process state)	حالة العملية (تكون أحد حالات العملية التي ذكرناها سابقاً)

أولويات العمليات PROCESS PRIORITY

- تحدّد أولوية العملية مقدار حصولها على وقت المعالج مقارنةً بالعمليات الأخرى.
- كلما ارتفعت أولوية العملية، حصلت على وقت أكبر من وحدة المعالجة المركزية (CPU) ، وفي نظام لينكس، أولوية أعلى تعني قيمة رقمية أقل.
- ويستخدم النظام قيمة nice للمساعدة في حساب أولوية العملية.

لا تمل, أكمل طريقك, إِنَّ اللَّهَ يُحِبُّ الْعَبْدَ الْأَحْوَجَ..

■ العلاقة بين الأولوية (Priority) وقيمة النيس (Nice Value)

المفهوم	الوصف	النطاق (range)	من يتحكم به
الأولوية (PR)	القيمة الفعلية التي يستخدمها النواة (Kernel) لجدولة العمليات	من 0 (أعلى أولوية) إلى 139 (أدنى أولوية)	النواة (Kernel)
NI (Nice Value)	تلميح يحدده المستخدم لتعديل أولوية العملية	من -20 (أعلى أولوية) إلى +19 (أدنى أولوية)	المستخدم أو المدير (admin)

حيث يتم حساب أولوية عمليات المستخدم اعتماداً على المعادلة التالية: $PR = 20 + NI$
وتكون قيمة الـ NI الافتراضية 0

👉 أمثلة:

$NI = -10 \rightarrow PR = 10 \rightarrow$ High priority

$NI = 0 \rightarrow PR = 20 \rightarrow$ Normal priority

$NI = +10 \rightarrow PR = 30 \rightarrow$ Low priority

هناك أوامر محددة تخوّلنا تغيير أولوية عملية محددة عن طريق تغيير قيمة الـ NI:

1. تشغيل عملية جديدة بقيمة Nice مُحددة: `nice -n <nice_value> <command>`
2. تغيير أولوية عملية تعمل بالفعل: `renice -n <nice_value> -p <PID>`
نحصل على الـ PID من تعليمة ps أو top

في كلا الأمرين السابقين لا يمكن إعطاء قيم للـ NI أقل أو يساوي الصفر إلا من قبل مستخدم مع صلاحيات الـ root.

مثال توضيحي: سنقوم بالتجربة على العملية ذات الرقم 2130

```

rbc@ -VMware-Virtual-Platform:~$ top
top - 10:52:24 up 1:31, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 289 total, 1 running, 288 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.2 us, 0.6 sy, 0.0 ni, 99.2 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3867.7 total, 2052.0 free, 1109.5 used, 965.7 buff/cache
MiB Swap: 3729.0 total, 3729.0 free, 0.0 used. 2758.3 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 2130 rbc      20   0 4001776 385220 142800 S   1.7   9.7   0:49.06 gnome-s+
    28 root      20   0      0      0      0 I   0.3   0.0   0:04.65 kworker+
 3489 rbc      20   0 553388 52356 41832 S   0.3   1.3   0:00.83 gnome-t+
 3511 rbc      20   0 14536 5848 3672 R   0.3   0.1   0:00.04 top
    1 root      20   0 23116 14020 9412 S   0.0   0.4   0:05.26 systemd
    2 root      20   0      0      0      0 S   0.0   0.0   0:00.03 kthreadd

```

نريد أن نجعل لها أولوية أقل، مما يعني أننا نريد زيادة قيمة ال PR أي تغيير قيمة ال NI لقيمة أعلى.
قمنا بتغيير قيمة NI لتصبح 5 بالتالي حسب العلاقة: $PR = 20 + 5$ أصبحت قيمة ال PR 25 وهي أكبر من القيمة السابقة
بالتالي أصبحت أولوية العملية أقل:

```
rbcs@ -VMware-Virtual-Platform:~$ renice -n 5 -p 2130
2130 (process ID) old priority 0, new priority 5
rbcs@ -VMware-Virtual-Platform:~$ top
```

```
top - 10:53:31 up 1:32, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 298 total, 1 running, 297 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.3 us, 1.5 sy, 0.0 ni, 98.0 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 3867.7 total, 2027.5 free, 1131.3 used, 968.5 buff/cache
MiB Swap: 3729.0 total, 3729.0 free, 0.0 used. 2736.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2130	rbcs	25	5	4001776	385180	142800	S	2.6	9.7	0:50.90	gnome-s+
3436	rbcs	20	0	3145824	61060	46660	S	0.7	1.5	0:00.77	gjs
27	root	20	0	0	0	0	S	0.3	0.0	0:00.24	ksoftir+
371	root	19	-1	67292	18108	16572	S	0.3	0.5	0:01.46	systemd+
3179	root	0	-20	0	0	0	I	0.3	0.0	0:00.07	kworker+
3364	root	20	0	0	0	0	I	0.3	0.0	0:00.08	kworker+
3489	rbcs	20	0	553524	52520	41996	S	0.3	1.3	0:01.00	gnome-t+
3600	rbcs	20	0	14536	5840	3664	R	0.3	0.1	0:00.06	top
1	root	20	0	23116	14020	9412	S	0.0	0.4	0:05.35	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.03	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_wo+
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker+

والآن نريد زيادة أولويتها مرة أخرى، أي يجب إنقاص قيمة PR مما يعني إعطاء قيمة NI أقل من القيمة الحالية.
سنقوم بتغيير قيمة NI لتصبح -7 فتصبح قيمة $PR = 13$ وهي أقل من القيمة السابقة، بالتالي أصبحت الأولوية أعلى.
نلاحظ، في الأمر الأول حاولنا تغيير قيمة ال NI لقيمة سالبة كمستخدم عادي وهو غير مسموح

```
rbcs@ -VMware-Virtual-Platform:~$ renice -n -7 -p 2130
renice: failed to set priority for 2130 (process ID): Permission denied
rbcs@ -VMware-Virtual-Platform:~$ sudo renice -n -7 -p 2130
[sudo] password for rbcs:
2130 (process ID) old priority 5, new priority -7
rbcs@ -VMware-Virtual-Platform:~$ top
```

```
top - 10:54:18 up 1:33, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 298 total, 1 running, 297 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 13.0 sy, 0.0 ni, 87.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3867.7 total, 2011.3 free, 1147.5 used, 968.6 buff/cache
MiB Swap: 3729.0 total, 3729.0 free, 0.0 used. 2720.2 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2130	rbcs	13	-7	4001760	385180	142800	S	18.2	9.7	0:52.42	gnome-s+
3352	root	20	0	0	0	0	I	9.1	0.0	0:00.27	kworker+
3609	rbcs	20	0	14536	5520	3344	R	9.1	0.1	0:00.03	top
1	root	20	0	23116	14020	9412	S	0.0	0.4	0:05.37	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.03	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_wo+

الإشارات في لينكس (Signals)

■ مفهوم الإشارات:

هي طريقة تستخدمها نواة لينكس أو أحد العمليات لإبلاغ عملية معينة بأن حدثاً ما قد وقع. تستخدم الإشارات لـ:

- إنهاء العمليات
- إيقاف/استئناف التنفيذ
- معالجة الأخطاء
- إعادة تحميل الإعدادات
- التواصل بين العمليات

ويستخدم الأمر kill لإرسال هذه الإشارات إلى العمليات، وبنيتها الأساسية من الشكل:

Kill [-signal] PID

حيث يمكن استخدام اسم الإشارة أو رقمها في الأمر وإن لم تحدّد إشارة، يتم إرسال الإشارة الافتراضية SIGTERM رقم 15.

Common Signals الإشارات الشائعة

الإشارة	رقم الإشارة	الوصف
SIGHUP	1	إشارة "تعليق الاتصال" — تستخدم غالباً لإخبار الخدمة بإعادة تحميل الإعدادات بدون إنهاء العملية.
SIGINT	2	إيقاف تفاعلي — (Interrupt) ترسل عند الضغط على Ctrl + C لإنهاء البرنامج بشكل طبيعي.
SIGQUIT	3	إيقاف مع تفريغ الذاكرة — (Quit) يؤدي إلى إنهاء العملية وكتابة ملف core dump للتصحيح.
SIGKILL	9	قتل فوري — (Kill) يُجبر العملية على الإنهاء فوراً دون إمكانية التعامل معه. لا يمكن تجاهله أو اعتراضه.
SIGTERM	15	إنهاء لطيف — (Terminate) الطلب من العملية أن تنهي نفسها بشكل منظم. الإشارة الافتراضية من kill .
SIGSTOP	19	إيقاف فوري للعملية — (Stop) لا يمكن تجاهله أو التعامل معه.
SIGCONT	18	استئناف عملية كانت موقوفة بواسطة SIGSTOP.
SIGCHLD	17	ترسل للأب عند انتهاء أحد العمليات الأبناء، كي يقوم بتنظيفها
SIGUSR1 / SIGUSR2	10 / 12	إشارات مخصصة للتطبيقات (User-defined)، تستخدم لسلوك مخصص حسب البرنامج.

مثال توضيحي:

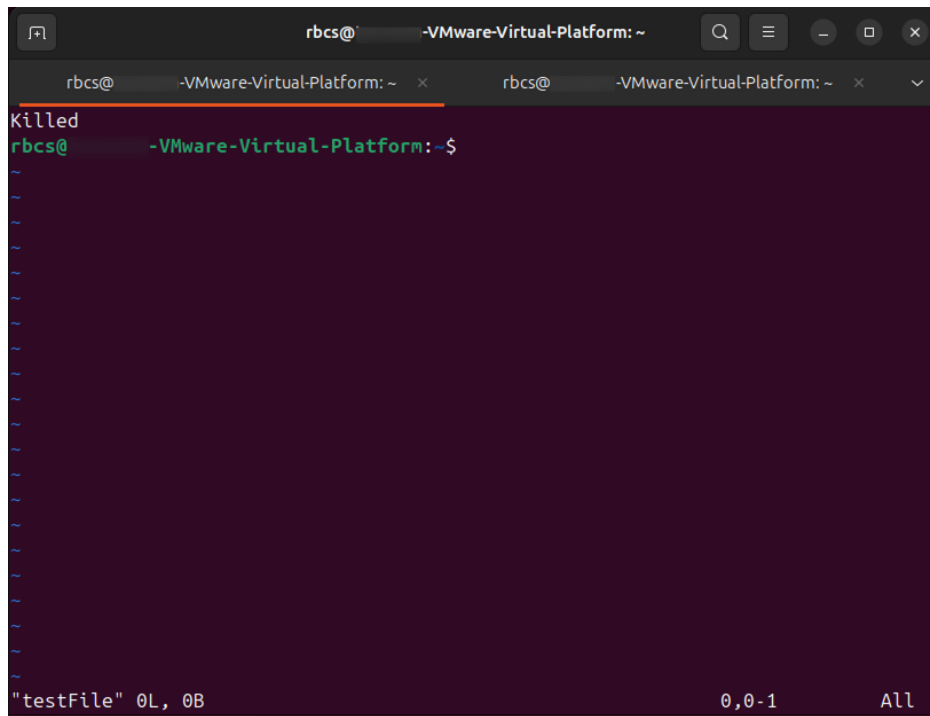
لدينا ملف testFile, قمنا بتشغيل الأمر vim testFile, حالياً ال vim ينتظر إدخال أو أمر بالحفظ والخروج أي أن العملية لا تزال تعمل ويمكننا رؤيتها ضمن تعليمة ps أو top.

قمنا بإنشاء terminal جديد لتجربة إيقاف العملية السابقة, ولإيقافها نحن بحاجة ل ID العملية, وكما ذكرنا سابقاً يمكننا إيجاده من تعليمة ps أو top.

بعدها نقوم بإيقاف العملية بشكل نهائي بالأمر: kill -9 2885 (2885 هو رقم العملية وليس رقم ثابت) (يمكن استخدام اسم الإشارة أيضاً ويجب أن يكتب بأحرف كبيرة: kill -SIGKILL 2885)

USER	PID	%CPU	%MEM	VSZ	SSZ	T	TIME	COMMAND
rbcs	2462	0.0	0.1	309792	6312	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2481	0.0	0.1	309772	6580	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2482	0.2	0.7	825440	29880	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2493	0.0	0.2	389376	7952	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2502	0.0	0.1	310760	6900	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2527	0.2	0.2	39136	11796	?	Ss 11:41	0:00 /snap/snapd-d
rbcs	2584	0.5	0.7	429624	30804	?	Sl 11:41	0:00 /snap/snapd-d
rbcs	2611	0.0	0.1	236768	7404	?	Sl 11:41	0:00 /usr/libexec/
rbcs	2629	0.0	0.1	230108	5924	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2638	0.0	0.1	236268	6264	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2639	0.0	0.2	535708	8864	?	Sl 11:41	0:00 /usr/libexec/
rbcs	2670	0.4	0.7	734624	29876	?	SNsl 11:41	0:00 /usr/libexec/
rbcs	2692	0.1	0.6	2728148	26908	?	Sl 11:41	0:00 /usr/bin/gjs
rbcs	2708	0.1	0.3	701608	14356	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2713	0.3	1.0	843604	40740	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2756	0.8	1.5	3145828	61548	?	Sl 11:41	0:00 gjs /usr/shar
rbcs	2793	0.2	0.6	417236	24920	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2824	1.3	1.3	554052	52572	?	Ssl 11:41	0:00 /usr/libexec/
rbcs	2831	0.0	0.1	11024	5140	pts/0	Ss 11:41	0:00 bash
rbcs	2885	0.1	0.3	27336	13032	pts/0	SL+ 11:41	0:00 vim testFile
rbcs	2896	0.0	0.1	11024	5032	pts/1	Ss 11:42	0:00 bash
rbcs	2903	100	0.1	13744	4560	pts/1	R+ 11:42	0:00 ps aux





وهكذا نصل إلى نهاية محاضرتنا، لكن تذكّروا أن بعض النهايات ليست إلا مهداً
لبدايات أروع وأعظم.....